Product Requirements Document (PRD)

Project: WASM-RPG (Adaptive Native-Speed Learning Engine)

Track: 2 (Intelligent Systems, AI & Gamified Experiences) **PS-ID:** PS-201

1. Product Overview

Current educational platforms gamify learning using superficial rewards (badges, points). WASM-RPG disrupts this by using "Mechanic-as-Metaphor." It evaluates a student's academic weaknesses via a web dashboard and dynamically injects those parameters into a natively compiled (C++ to WebAssembly) 16-bit RPG. The student must play through dungeons where the physical rules of the game are governed by the concepts they failed to grasp.

2. Core Architecture

The system is fully decoupled into three high-performance tiers:

- **Tier 1: React PWA (The Shell)**
  - Serves as the student dashboard.
  - Administers rapid-fire diagnostic quizzes.
  - Hosts the HTML5 <canvas> element that runs the WebAssembly game binary.
- **Tier 2: FastAPI & Python (The AI Brain)**
  - Analyzes diagnostic results.
  - Generates a structured JSON payload representing the generated level (e.g., enemy types, maze logic, puzzle parameters).
- **Tier 3: C++ & WebAssembly (The Game Engine)**
  - A lightweight 2D engine built in C++ (using Raylib or SDL2).
  - Compiled via Emscripten to .wasm.
  - Reads the backend's JSON payload on boot and procedurally generates the level to match the student's learning gap.

3. Minimum Viable Product (MVP) for 24-Hour Hackathon

To win a 24-hour hackathon, **do not try to build an entire game**. Build the pipeline and *one* fully functional level mechanic to prove the concept.

The Prototype Goal: "The Data Structures Boss Room"

- **UI:** A simple React screen with 3 questions about Stacks (LIFO) vs. Queues (FIFO).
- **Backend:** If the user fails the "Stacks" question, FastAPI returns: {"boss_type": "stack", "minion_count": 5}.
- **Game:** The WASM engine loads. The player faces a boss holding 5 minions stacked vertically. The player's sword attack only registers if they hit the top minion first. Hitting the bottom reflects damage.

4. Hackathon 24-Hour Execution Roadmap (April 10-11)

- **Hours 0-4 (Setup):** * Member 1: Setup React + Vite PWA boilerplate.
    - Member 2: Setup FastAPI + SQLite backend.
    - Member 3: Setup C++ Emscripten pipeline, compile a blank window to <canvas>.
- **Hours 4-12 (Core Logic):**
    - Member 1 & 2: Build the quiz UI and JSON payload generator.
    - Member 3: Build the basic player movement and collision in C++.
- **Hours 12-18 (Integration - The Hard Part):**
    - Pass the JSON payload from the React frontend into the running WASM module.
    - Implement the "Stack" boss combat logic.
- **Hours 18-24 (Polish):**
    - Add pixel art assets (use free itch.io assets, do not draw your own).
    - Prepare the final pitch and live demonstration.